
 CHALLENGE 2 - SINGULAR VALUE DECOMPOSITION AND IMAGE COMPRESSION

Goal: Apply the singular value decomposition to image compression and noise reduction.

Data: Download the file `einstein.jpg` (256px) and move it to your working directory.

Overview

Any $m \times n$ matrix A possesses a **singular value decomposition** (SVD) of the form:

$$A = U\Sigma V^T;$$

where U is an orthogonal $m \times m$ matrix satisfying the condition $U^T U = I_m$, Σ is an $m \times n$ rectangular diagonal matrix with nonnegative values $\sigma_1, \sigma_2, \dots, \sigma_p$, with $p = \min(m, n)$, on the main diagonal, and V is an orthogonal $n \times n$ matrix satisfying the condition $V^T V = I_n$. The singular values are arranged in the decreasing order: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$. The singular value decomposition of a matrix is **not unique**.

Singular value decomposition is important for the implementation of many numerical algorithms. In this project we will look at two applications related to image processing and noise filtering. We remark that other more efficient image compression methods exist. The examples in this project are used mainly to show the reduction in the amount of data that can be accomplished with SVD.

- The SVD can be used to express A in the form

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \dots + \sigma_r \mathbf{u}_r \mathbf{v}_r^T,$$

where r is the rank of A and $\mathbf{u}_i$ and $\mathbf{v}_i$ are the i th columns of U and V respectively. Observe that since the singular values are arranged in the decreasing order, the first terms in this sum provide a larger contribution than the subsequent terms.

- The truncated SVD considering the first k terms, with $k < r$ is the best approximation of the matrix A among the matrices of the rank at most k in the sense of Frobenius norm. This can be used for image compression as follows: instead of storing the whole $m \times n$ matrix A , we can instead store the $m \times k$ and $n \times k$ matrices

$$C = [\mathbf{u}_1 \ \mathbf{u}_2 \ \dots \ \mathbf{u}_k], \quad D = [\sigma_1 \mathbf{v}_1 \ \sigma_2 \mathbf{v}_2 \ \dots \ \sigma_k \mathbf{v}_k]. \quad (1)$$

If k is much smaller than p , then storing C and D will take much less space than storing the full matrix A . The compressed image can be simply computed as $\tilde{A} = CD^T$.

Tasks

- Load the image as an **Eigen** matrix A with size $m \times n$. Each entry in the matrix corresponds to a pixel on the screen and takes a value somewhere between 0 (black) and 255 (white). Compute the matrix product $A^T A$ and report the euclidean norm of $A^T A$.

```
Both the following answers are correct:  
Euclidean norm: 1.0458e+09 (1.6083e+04 if A is scaled with coeffs between 0 and 1)  
Frobenius norm: 1.05041e+09 (1.61539e+04 if A is scaled with coeffs between 0 and 1)
```

- Solve the eigenvalue problem $A^T A \mathbf{x} = \lambda \mathbf{x}$ using the proper solver provided by the **Eigen** library. Report the two largest computed singular values of A .

```
Two largest singular values of A:  
32339.1  
9523.11  
  
If the matrix is scaled:  
126.82  
37.3456
```

- Export matrix $A^T A$ in the matrix market format and move it to the `lis-2.1.6/test` folder. Using the proper iterative solver available in the LIS library compute the largest eigenvalue of $A^T A$ up to a tolerance of 10^{-8} . Report the computed eigenvalue. Is the result in agreement with the one obtained in the previous point?

```
Largest eigenvalue of A^T A is 1.04582e+09 (1.60833e+04 if the matrix is scaled),  
computed in 8 iterations of the power method.  
Yes, the results are in agreement since the computes eigenvalue of A^T A is the  
square of the largest singular value computed previously.
```

- Find a **shift** $\mu \in \mathbb{R}$ yielding an acceleration of the previous eigensolver. Report μ and the number of iterations required to achieve a tolerance of 10^{-8} .

```
Power method: shift = 4.0e+07  
Power method: eigenvalue = 1.045818e+09  
Power method: number of iterations = 7
```

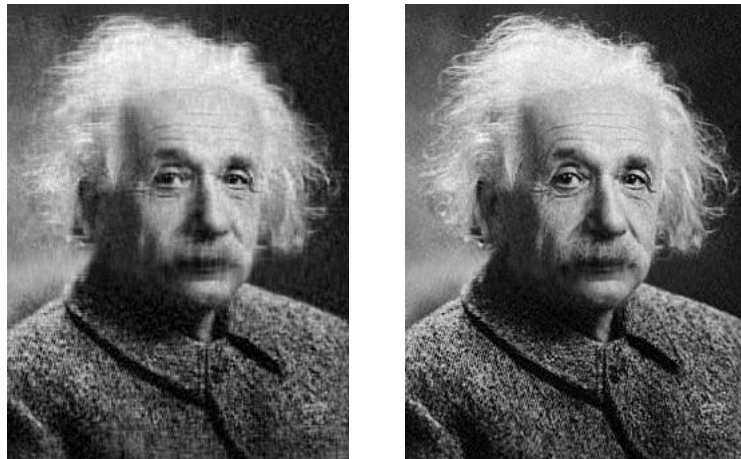
- Using the SVD module of the **Eigen** library, perform a singular value decomposition of the matrix A . Report the Euclidean norm of the diagonal matrix Σ of the singular values.

```
35576.6 (139.516 if the matrix is scaled)
```

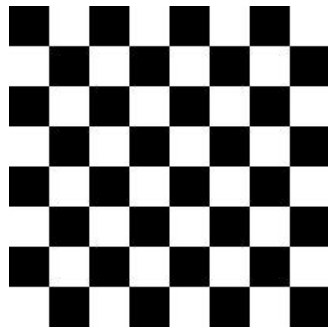
- Compute the matrices C and D described in (1) assuming $k = 40$ and $k = 80$. Report the number of nonzero entries in the matrices C and D .

```
For k=40: nonzero entries in C= 13640, nonzero entries in D= 10240  
For k=80: nonzero entries in C= 27280, nonzero entries in D= 20480
```

- Compute the compressed images as the matrix product CD^T (again for $k = 40$ and $k = 80$). Export and upload the resulting images in .png.

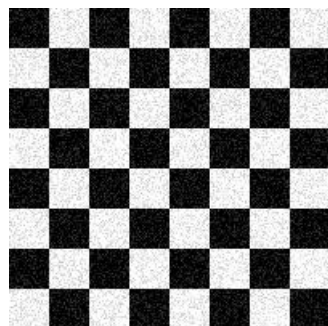


- Using **Eigen** create a black and white checkerboard image (as the one depicted below) with height and width equal to 200 pixels. Report the Euclidean norm of the matrix corresponding to the image.



36062.2 (141.42 **if** the matrix is scaled with coeffs between 0 and 1)

- Introduce a noise into the checkerboard image by adding random fluctuations of color ranging between $[-50, 50]$ to each pixel. Export the resulting image in .png and upload it.



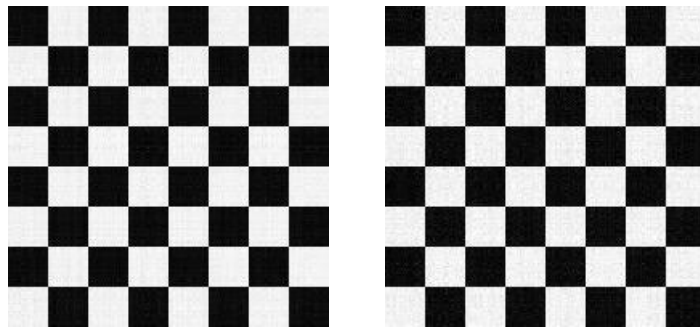
- Using the SVD module of the **Eigen** library, perform a singular value decomposition of the matrix corresponding to the noisy image. Report the two largest computed singular values.

Two largest singular values (this depends on the particular noise randomization):
 25523.29. (100.091 **if** the matrix is scaled)
 22984.3. (90.1345 **if** the matrix is scaled)

- Starting from the previously computed SVD, create the matrices C and D defined in (1) assuming $k = 5$ and $k = 10$. Report the size of the matrices C and D .

For $k=5$: size of $C= 200 \times 5$, size of $D= 200 \times 5$
 For $k=10$: size of $C= 200 \times 10$, size of $D= 200 \times 10$

- Compute the compressed images as the matrix product CD^T (again for $k = 5$ and $k = 10$). Export and upload the resulting images in **.png**.



- Compare the compressed images with the original and noisy images. Comment the results.

For the first image, we can observe that the truncated SVD allows to compress the original images by storing around 72 percent (resp. 45 percent) less information with $k=40$ (resp. $k=80$).

For the second image, the truncated SVD also provides a denoising strategy that allow to store 95 (resp. 90) percent less information with $k=5$ (resp. $k=10$).